

Blatt 03 – Calculator: Anonyme Klassen und Lambda-Ausdrücke

Modul: Programmieren 2

Adrian Kramkowski

Datum: 13.05.2026

1. Projekt klonen und Gradle konfigurieren

Zunächst wurde das Calculator-Projekt lokal geklont bzw. geforkt und in die IDE geladen. Anschließend wurde Gradle als Build-System eingerichtet, damit das Projekt kompiliert, gestartet und später erweitert werden kann.

```
git clone CALCULATOR_REPOSITORY_URL
cd calculator
```

Ziel ist die Erweiterung des vorhandenen Rechners um zusätzliche Operationen unter Nutzung verschiedener Java-Konzepte:

- Normale Klasse

- Anonyme Klasse
- Lambda-Ausdruck
- Lambda für Event-Handling

2. Analyse des Interfaces Operation

Das Interface `Operation` definiert genau eine Methode:

```
int doOperation(int a, int b);
```

Dadurch handelt es sich funktional um ein Functional Interface, sodass neben normalen Klassen auch anonyme Klassen und Lambda-Ausdrücke eingesetzt werden können.

3. Neue Klasse Sub (Subtraktion)

Für die Subtraktion wurde eine eigene Java-Klasse erstellt, die das Interface `Operation` implementiert.

```
package calculator;

/** Simple subtraction of two integers. */
public class Sub implements Operation {
```

```
@Override
public int doOperation(int a, int b) {
    return a - b;
}
}
```

Einbindung in Calculator:

```
operations.put("Sub", new Sub());
```

Diese Lösung erfüllt die Vorgabe, keine anonyme Klasse und keinen Lambda-Ausdruck zu verwenden.

4. Multiplikation Mul als anonyme Klasse

Die Multiplikation wurde direkt innerhalb von `setupOperationSelector()` als anonyme Klasse umgesetzt:

```
operations.put(
    "Mul",
    new Operation() {
        @Override
        public int doOperation(int a, int b) {
            return a * b;
        }
    });
```

Dadurch ist keine separate Datei notwendig, die Implementierung bleibt jedoch explizit objektorientiert.

5. Division Div als Lambda-Ausdruck

Für die Division wurde ein Lambda-Ausdruck genutzt:

```
operations.put("Div", (a, b) -> a / b);
```

Diese Schreibweise ist besonders kompakt und eignet sich, da `operation` nur eine abstrakte Methode besitzt.

6. ActionListener von anonymer Klasse zu Lambda

Der bestehende ActionListener der JComboBox wurde von einer anonymen Klasse in einen Lambda-Ausdruck überführt:

```
operationSelector.addActionListener(  
    e -> {  
        try {  
            result.setText("" + calculate());  
        } catch (NumberFormatException ex) {  
            System.out.println("Invalid input.");  
        }  
    }  
);
```

```
    }  
});
```

Vorteile:

- Weniger Code
- Bessere Lesbarkeit
- Modernere Java-Syntax

7. Vollständige Methode setupOperationSelector()

```
private void setupOperationSelector() {  
    operations = new HashMap<>();  
    operations.put("Add", new Add());  
  
    operations.put("Sub", new Sub());  
  
    operations.put(  
        "Mul",  
        new Operation() {  
            @Override  
            public int doOperation(int a, int b) {  
                return a * b;  
            }  
        }  
    );  
});
```

```
operations.put("Div", (a, b) -> a / b);

operationSelector = new JComboBox<>();
operations.forEach((key, value) -> operationSelector.addItem(key));

operationSelector.addActionListener(
    e -> {
        try {
            result.setText("" + calculate());
        } catch (NumberFormatException ex) {
            System.out.println("Invalid input.");
        }
    });
}
```

8. Praktische Erkenntnis

Diese Aufgabe verdeutlicht die Entwicklung von klassischer objektorientierter Implementierung hin zu moderner funktionaler Syntax in Java:

- **Normale Klasse:** maximale Struktur
- **Anonyme Klasse:** kompakter ohne separate Datei
- **Lambda:** minimalistisch und modern

Dadurch wird deutlich, wann welche Implementierungsform sinnvoll ist und wie sich Java im Bereich funktionaler Programmierung erweitert hat.

9. Zusammenfassung

Der Calculator wurde erfolgreich um Subtraktion, Multiplikation und Division erweitert. Zusätzlich wurde das Event-Handling modernisiert. Die Aufgabe zeigt praxisnah den Unterschied zwischen klassischen Klassen, anonymen Klassen und Lambda-Ausdrücken und stärkt das Verständnis für moderne Java-Entwicklung.